Homework 5

AUTHOR
YOUR NAME HERE

This homework will involve data from transcripts of the American television show "The Office" which aired on NBC from 2005 to 2013. All transcripts are available in the R package schrute, which is available on CRAN.

```
library(schrute)
```

```
Warning: package 'schrute' was built under R version 4.3.2
```

```
library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.3.2

── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.1.2     ✓ readr     2.1.4
✓ forcats   1.0.0     ✓ stringr   1.5.0
✓ ggplot2   3.4.3     ✓ tibble    3.2.1
✓ lubridate 1.9.2     ✓ tidyr     1.3.0
✓ purrr     1.0.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidytext)
```

```
Warning: package 'tidytext' was built under R version 4.3.2
```

```
data(theoffice)
glimpse(theoffice)
```

```
Rows: 55,130
Columns: 12
$ index            <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16…
$ season           <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,…
$ episode          <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,…
$ episode_name     <chr> "Pilot", "Pilot", "Pilot", "Pilot", "Pilot", "Pilot",…
$ director         <chr> "Ken Kwapis", "Ken Kwapis", "Ken Kwapis", "Ken Kwapis…
$ writer           <chr> "Ricky Gervais;Stephen Merchant;Greg Daniels", "Ricky…
$ character        <chr> "Michael", "Jim", "Michael", "Jim", "Michael", "Micha…
$ text             <chr> "All right Jim. Your quarterlies look very good. How …
$ text_w_direction <chr> "All right Jim. Your quarterlies look very good. How …
$ imdb_rating      <dbl> 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6…
$ total_votes      <int> 3706, 3706, 3706, 3706, 3706, 3706, 3706, 3706, 3706,…
$ air_date         <chr> "2005-03-24", "2005-03-24", "2005-03-24", "2005-03-24…
```

Note that each row in the data frame corresponds to a line spoken by a character in the show.

Problem 1 [60 Points Total]

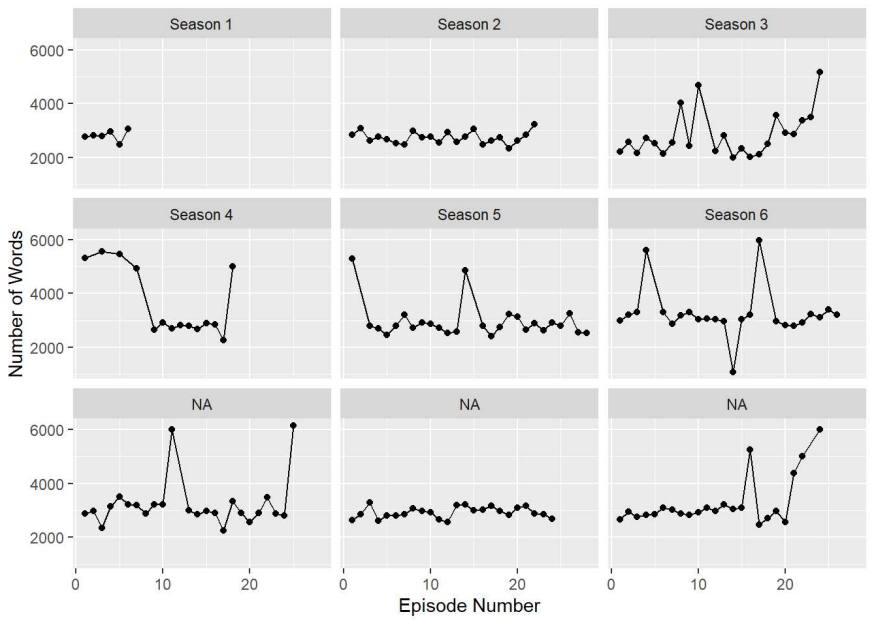
Part A [10 Points]

The column named text in theoffice contains the words spoken by a character in a line of dialog. Recreate the following figure showing the total number of words spoken in each episode of each season.

```
# INSERT SOLUTION HERE
# Load the data
data(theoffice)

# Unnest the text into words
theoffice_longer <- theoffice %>%
  unnest_tokens(word, text)

# Count the number of words for each episode in each season
df_counts <- theoffice_longer %>%
  group_by(season, episode) %>%
  summarise(counts = n())
```

```
`summarise()` has grouped output by 'season'. You can override using the
`.groups` argument.
```

```r
# Plotting the number of words per episode for each season
ggplot(df_counts, aes(x = episode, y = counts)) +
  geom_line() +
  geom_point() +
  labs(x = "Episode Number", y = "Number of Words")+
  facet_wrap(~season, labeller = as_labeller(c(`1` = "Season 1", `2` = "Season 2", `3` = "Season 3", `4` =
```



Based on the number of words, list out the episodes do you think might be "double episodes" (42 minute instead of 21 long)?

```r
# INSERT SOLUTION HERE
# Assuming that a "double episode" has at least 1.5 times the average word count
double_threshold <- mean(df_counts$counts) * 1.5
double_episodes <- df_counts %>%
  filter(counts > double_threshold)

# Check for consecutive episode numbers
double_episodes %>%
  rowwise() %>%
  mutate(followed_by_next_episode = ((season == lag(season) & episode == (lag(episode) - 1)) |
                                     (season == lead(season) & episode == (lead(episode) + 1)))) %>%
  filter(!followed_by_next_episode)
```

```
# A tibble: 0 × 4
# Rowwise:  season
# ℹ 4 variables: season <int>, episode <int>, counts <int>,
#   followed_by_next_episode <lgl>
```

```r
double_episodes
```

```
# A tibble: 16 × 3
# Groups:   season [6]
   season episode counts
    <int>   <int>  <int>
 1      3      10   4679
 2      3      24   5165
 3      4       1   5307
 4      4       3   5568
 5      4       5   5460
 6      4       7   4928
 7      4      18   4996
 8      5       1   5293
 9      5      14   4867
10      6       4   5614
11      6      17   5973
12      7      11   6010
13      7      25   6160
14      9      16   5261
15      9      22   5006
16      9      24   6012
```

Verify your claims by showing that none of these episodes have an episode number right after them. (Example, there is no Season 4 Episode 2, confirming Season 4 Episode 1 is a double episode)

```
# INSERT SOLUTION HERE

library(dplyr)

# Load the data
data("theoffice")

# Group by season and then check for consecutive episode numbers
theoffice %>%
  select(season, episode) %>%
  distinct() %>%
  arrange(season, episode) %>%
  group_by(season) %>%
  mutate(next_episode_exists = lead(episode) == episode + 1) %>%
  filter(next_episode_exists == FALSE) %>%
  ungroup()
```

```
# A tibble: 11 × 3
   season episode next_episode_exists
    <int>   <int> <lgl>
 1      3      10 FALSE
 2      4       1 FALSE
 3      4       3 FALSE
 4      4       5 FALSE
 5      4       7 FALSE
 6      5       1 FALSE
 7      5      14 FALSE
 8      6       4 FALSE
 9      6      17 FALSE
10      7      11 FALSE
11      9      22 FALSE
```

Finally, guess which episode (hint: in Season 6) was a clip show. Feel free to verify using wikipedia

```
# INSERT SOLUTION HERE
# Identify the episode with significantly fewer words, suggesting a clip show
clip_show_threshold <- mean(df_counts$counts) - 2 * sd(df_counts$counts)
clip_show_episode <- df_counts %>%
  filter(counts < clip_show_threshold & season == 6)

clip_show_episode
```

```
# A tibble: 1 × 3
# Groups:   season [1]
  season episode counts
   <int>   <int>  <int>
1      6      14   1088
```

Part B [8 Points]

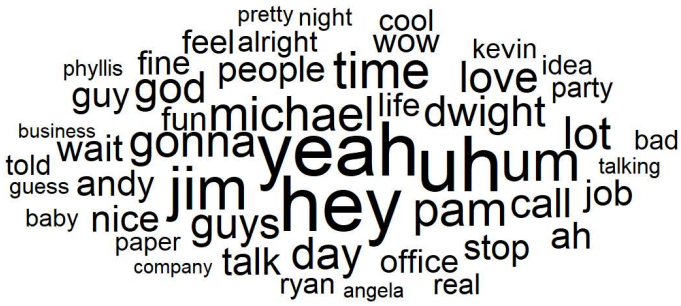
Create a word cloud of the 50 most-used words across the entire series, excluding stop words.

```
# INSERT SOLUTION HERE

library(ggwordcloud)
```

```
Warning: package 'ggwordcloud' was built under R version 4.3.2
```

```
data("theoffice")


# Unnest the text into a longer format, one row per word
theoffice_longer <- theoffice %>%
  unnest_tokens(word, text) # This converts text to a tidy format where each word is on a new row
# You may need to load the 'stop_words' dataset from the tidytext package
data("stop_words")

# Filter out stop words from the unnested text
filtered_words <- theoffice_longer %>%
  anti_join(stop_words, by = "word")
theoffice_counts <- filtered_words %>%
  count(word, sort = TRUE)
theoffice_counts %>%
  top_n(50, wt = n) %>%
  ggplot(aes(label = word, size = n)) +
```

```
  geom_text_wordcloud_area() +
  scale_size_area(max_size = 30) +
  theme_minimal()
```



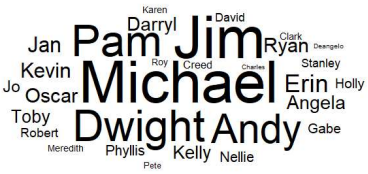
What word was used the most?

Yeah - the largest word in the wordcloud

Part C [4 Points]

Recreate the following word cloud of names of the 30 characters from the office who spoke the most words throughout the series. The size of the name should be proportional to how many words [including stop words] that the character spoke.

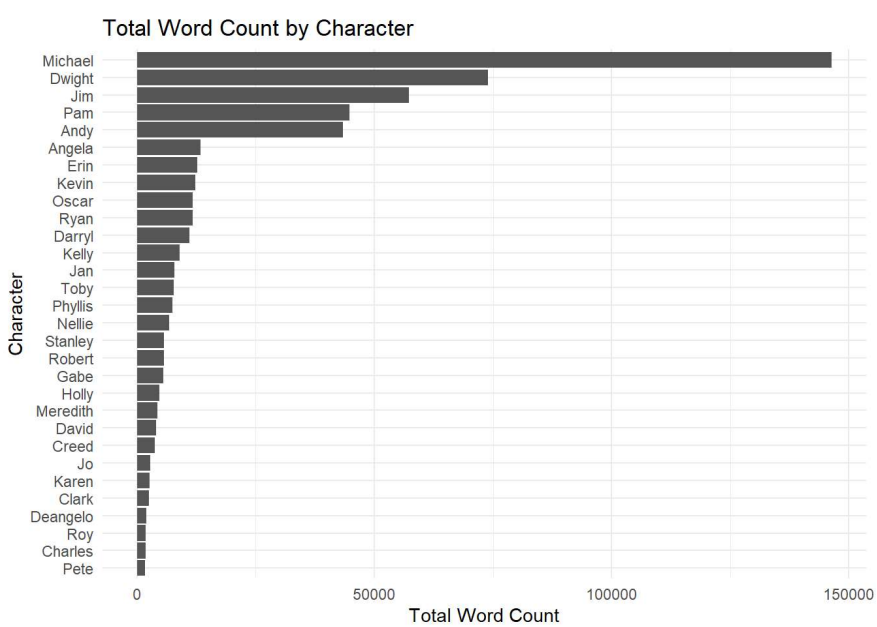
```
# INSERT SOLUTION HERE

data(theoffice)

# Aggregate total word counts by character
character_word_counts <- theoffice %>%
  group_by(character) %>%
  summarize(total_words = sum(str_count(text, "\\S+")))

# Sort and select the top 30 characters
top_characters <- character_word_counts %>%
  arrange(desc(total_words)) %>%
  slice_head(n = 30)

# Use ggplot2 and ggwordcloud to create the word cloud
library(ggwordcloud)
ggplot(top_characters, aes(label = character, size = total_words)) +
  geom_text_wordcloud_area() +
  scale_size_area(max_size = 30) +
  theme_minimal()
```



Part D [8 Points]

Consider the names of the 30 characters chosen for Part C. Recreate the following bar chart. It shows how many times that each character's name appeared as spoken dialog throughout the show's run.

```
#INSERT SOLUTION HERE
library(ggplot2)

# Assuming you have already aggregated the total word counts by character and selected the top 30 characte
# If not, you can use the code you provided to do so

# Plotting the bar chart
ggplot(top_characters, aes(x = reorder(character, total_words), y = total_words)) +
  geom_bar(stat = "identity") +
  coord_flip() +  # to make the bars horizontal
  labs(x = "Character", y = "Total Word Count", title = "Total Word Count by Character") +
  theme_minimal()
```



Part E [10 Points]

Let's see if there are other common first names that are spoken throughout the show.

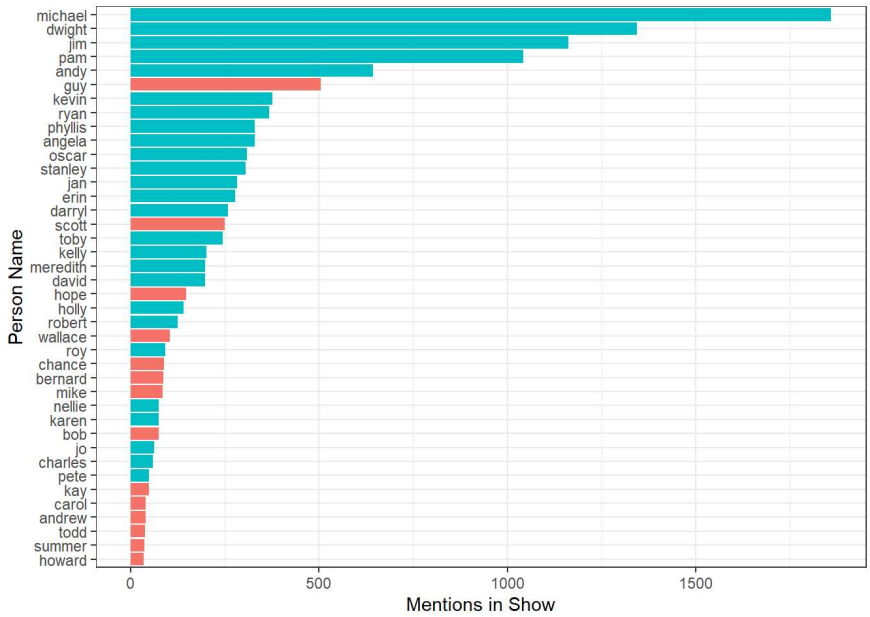
To do this, determine the 1500 most popular names in the babynames dataset [package babynames], remove any names that are also stop words (like Will and May), then count how times each of these names are spoken in the office.

Once you've done that, use the results to recreate the following plot. Each bar is colored based on whether or not the names appeared in Part D.

```
library("babynames")
```

```
Warning: package 'babynames' was built under R version 4.3.2
```

```
data(babynames)

character_data<-theoffice|>
  select(season,episode,text,character)|>
  unnest_tokens(word, text)|>
  summarize(term_frequency = n(),
            .by = c(character))


top_30_character_data<-character_data|>
  slice_max(term_frequency, n = 30)


top_30_characters<-
  data.frame(character=top_30_character_data$character,helper=1)|>
    mutate(character=tolower(character))


popular_1500_babynames<-babynames|>
  mutate(word=tolower(name))|>
  anti_join(stop_words,
            by = join_by(word))|>
  group_by(word)|>
  summarise(popularity=sum(n))|>
  slice_max(popularity,n=1500)

words_data<-theoffice|>
  select(season,episode,text,character,writer)|>
  unnest_tokens(word, text)



popular_names_office<-inner_join(words_data,popular_1500_babynames,by=c("word"="word"))|>
  group_by(word)|>
  summarise(value=n())|>
  slice_max(value,n=40)|>
  left_join(top_30_characters,by=c("word"="character"))|>
  mutate(present_in_top30 = if_else(is.na(helper), "No", "Yes"))



popular_names_office$word <- factor(popular_names_office$word, levels = popular_names_office$word[order(pc

  popular_names_office|>
    ggplot(mapping=aes(x = word, y = value,fill=present_in_top30)) +
  geom_bar(stat = "identity") +
    coord_flip() +
  theme_bw()+
  labs(x='Person Name',y='Mentions in Show')+
  theme(legend.position = "none")
```



Which character names shown in part D are missing from our list of common baby names?

```
# INSERT SOLUTION HERE
popular_names_office|>
  filter(present_in_top30=='No')|>
  mutate(missing_names=word)|>
   select(missing_names)
```

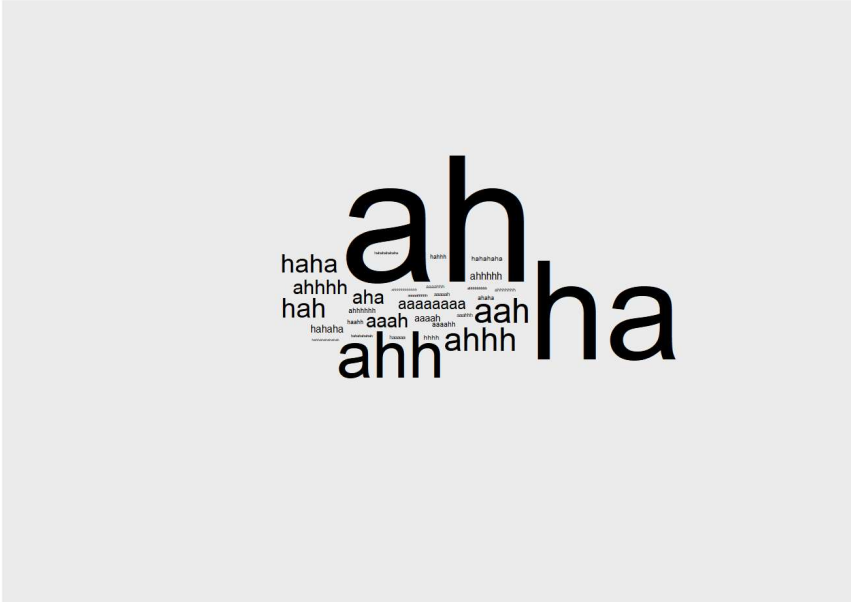
```
# A tibble: 14 × 1
   missing_names
   <fct>
 1 guy
 2 scott
 3 hope
 4 wallace
 5 chance
 6 bernard
 7 mike
 8 bob
 9 kay
10 andrew
11 carol
12 todd
13 summer
14 howard
```

Part F [6 Points]

In the office script, the writers often like to spell a character screaming or laughing using words that are at least two letters long and composed entirely of h's and a's. Approximately recreate the following word cloud demonstrating all of these words, with size proportional often they are used.

```
# INSERT SOLUTION HERE
words_data|>
  anti_join(stop_words)|>
  filter(str_detect(word,"^[h|a]+$"))|>
  group_by(word)|>
  summarise(term_frequency=n())|>
  ggplot() +
  geom_text_wordcloud_area(aes(label = word,
                               size = term_frequency)) +
  scale_size_area(max_size = 50)
```

```
Joining with `by = join_by(word)`
```

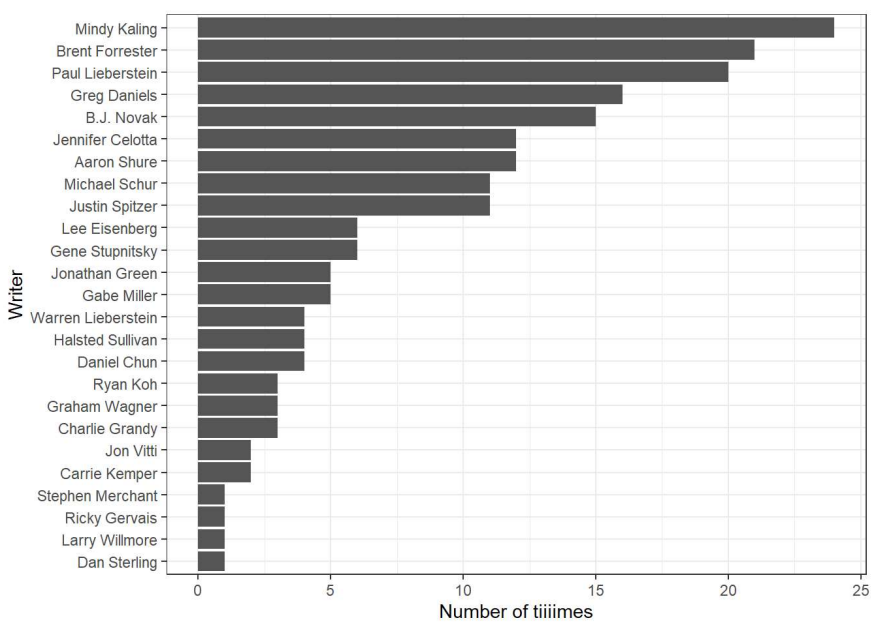


Part G [8 points]

Sometimes, the writers like to indicate yelling by showing the same vowel repeated four or more times in a row (e.g. aaaa, iiii, oooo) in a word. For example, "staaaarrrrted".

Recreate the following plot depicting how times this occurs in episodes written by each writer.

```
# INSERT SOLUTION HERE
writer_data<-words_data|>
  separate_longer_delim(writer,";")|>
    filter(str_detect(word,"aaaa+|eeee+|iiii+|oooo+|uuuu+|yyyy+"))|>
  group_by(writer)|>
  summarize(term_frequency=n())

writer_data$writer <- factor(writer_data$writer, levels = writer_data$writer[order(writer_data$term_freque

writer_data|>
  ggplot(mapping=aes(x = writer, y = term_frequency)) +
  geom_bar(stat = "identity") +
    coord_flip() +
  theme_bw()+
  labs(x='Writer',y='Number of tiiiimes')+
  theme(legend.position = "none")
```



Part H [6 points]

Treating each season as a document, calculate the tf_idf scores for the words. Which word has the highest tf_idf score for each season?

```
# INSERT SOLUTION HERE
words_data|>
  summarise(term_frequency = n(),
            .by = c(season, word))|>
  bind_tf_idf(word,
              season,
              term_frequency)|>
  arrange(desc(tf_idf)) |>
  group_by(season) |>
  slice_max(tf_idf, n = 1) |>
  select(season, word, tf_idf)
```

```
# A tibble: 9 × 3
# Groups:   season [9]
  season word         tf_idf
   <int> <chr>         <dbl>
1      1 downsizing 0.00143
2      2 chili's    0.000401
3      3 wimowheh   0.000907
4      4 rabies     0.000993
5      5 na         0.000702
6      6 sabre      0.000622
7      7 deangelo   0.00130
8      8 andy       0.000652
9      9 lice       0.000987
```

Problem 2 [40 Points Total]

Part A [8 Points]

Create a document term matrix corresponding to the number of times each character speaks in each episode. Each character name (e.g. Michael, Dwight, Pam) should correspond to a "term". Each episode corresponds to a "document".

```
# INSERT SOLUTION HERE


theoffice_unnested <- theoffice |>
  unnest_tokens(word, text)


missing <- anti_join(theoffice, theoffice_unnested,
          by = "character") |>
  select(character, text, text_w_direction)
missing
```

```
# A tibble: 16 × 3
   character                      text  text_w_direction
   <chr>                          <chr> <chr>
 1 Kevin's computer               ""    "[monkey noises]"
 2 Warehouse guys                 ""    "[kissing noises, sheep baaing sounds]"
 3 Men                            ""    "[generalized clapping]"
 4 Angela and Dwight              ""    "[silently working] [Pam walks back in…
 5 Dwight, Jim & Pam              ""    "[stare blankly at Michael]"
 6 Meredith, Creed, Oscar and Matt ""    "[cheering Michael]"
 7 Michael and Donna              ""    "[indistinct chatter inside conference…
 8 Mexican Man                    ""    "[speaking Spanish]"
 9 Mexican Man                    ""    "[speaking Spanish]"
10 Mexican Man                    ""    "[speaking Spanish]"
11 Mexican Man                    ""    "[speaking Spanish]"
12 Erin and Irene                 ""    "[screaming as Andy busts out of box]"
13 Guys in breakroom              ""    "[laughter]"
14 Pam & Dwight                   ""    "[laughs]"
15 Promo Announcer                ""    "[Speaks Danish]"
16 Promo Announcer                ""    "[Speaks Danish]"
```

```
length(unique(theoffice$character)) - length(unique(theoffice_unnested$character))
```

```
[1] 12
```

```
theoffice$text[theoffice$text == ""] <- NA

dtm<-theoffice |>
  unnest_tokens(word, text) |>
  summarise(term_frequency = n(),
            .by = c(character,season,episode)) |>
  mutate(season_episode = str_c(season, episode, sep = "_"))|>
  cast_dtm(document = season_episode,
           term = character,
           value = term_frequency)
dtm
```

```
<<DocumentTermMatrix (documents: 186, terms: 773)>>
Non-/sparse entries: 4006/139772
Sparsity           : 97%
Maximal term length: 33
Weighting          : term frequency (tf)
```

Hint: Your matrix should consist of 186 documents and 773 terms.

Part B [8 Points]

Use latent Dirichlet allocation to fit a topic model with four topics to this data.

```
# INSERT SOLUTION HERE
# INSERT SOLUTION HERE
library(topicmodels)
```

```
Warning: package 'topicmodels' was built under R version 4.3.2
```

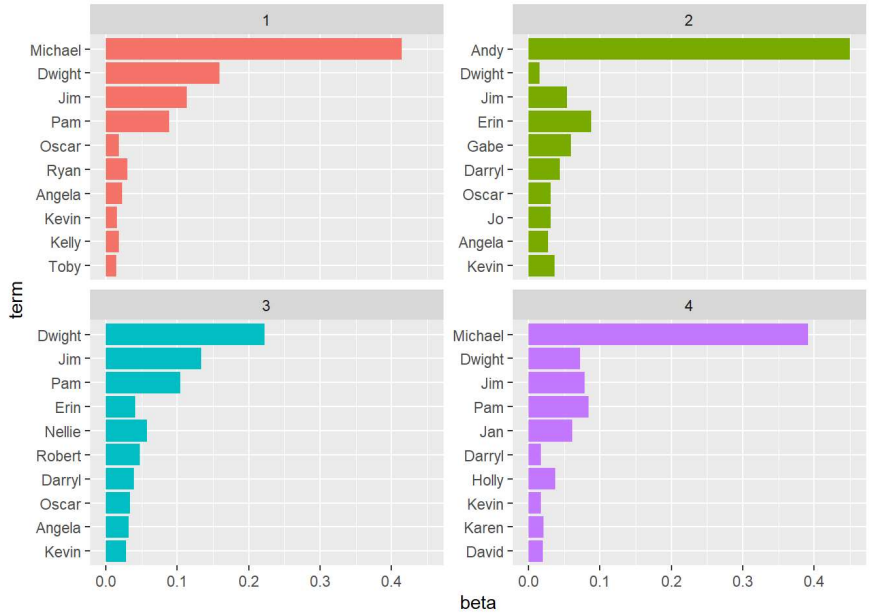
```
lda <- dtm |>
  LDA(k = 4, control = list(seed = 1))

class(lda)
```

```
[1] "LDA_VEM"
attr(,"package")
[1] "topicmodels"
```

Part C [12 Points]

Create a graphic showing the beta scores for the top 10 characters for each topic.

```
# INSERT SOLUTION HERE
lda |>
  tidy(matrix = "beta") |>
  group_by(topic) |>
  slice_max(beta,
            n = 10) |>
  mutate(topic = factor(topic))|>
  ggplot(aes(beta,
             fct_reorder(term,beta),
             fill = topic)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic,
             scales = "free_y",
             nrow = 2) +
  labs( y="term")
```



Is there a character whose prevalence seems to shift the most between topics?

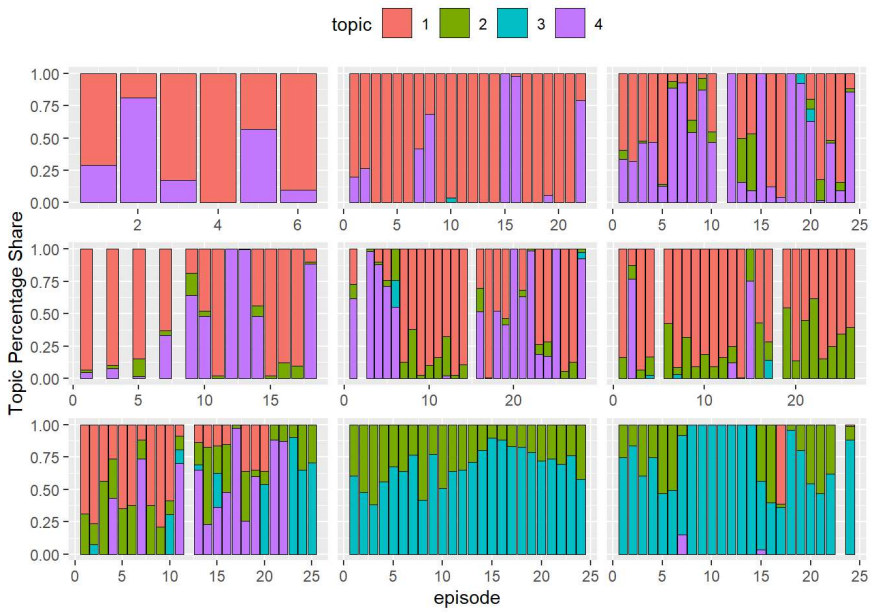
ANSWER HERE

Andy excels primarily in Topic 2, while his presence is less pronounced in other topics. Conversely, Michael is prominent in Topics 1 and 4 but lacks prominence in Topics 2 and 3.

Part D [12 Points]

Create a graphic showing the distribution of topics for each episode across seasons.

```
# INSERT SOLUTION HERE
tidy(lda, matrix = "gamma") |>
  separate_wider_delim(document,
                       delim = "_",
                       names = c("season",
                                 "episode")) |>
  mutate(season = parse_number(season),
         episode = parse_number(episode),
         topic = as_factor(topic)) |>
  arrange(topic) |>
  ggplot() +
  geom_col(aes(x = episode,
               y = gamma,
               fill = topic),
           position = "fill",
           color= "black",
           linewidth = 0.1) +
  facet_wrap(~season,
             scales = "free_x") +
  theme(legend.position = "top",
    strip.background = element_blank(),
    strip.text.x = element_blank()) +
  labs(y= "Topic Percentage Share")
```



Does the topic distributions seem to shift as the series progresses? How so? What do you think is driving this change?

ANSWER HERE.

Reasons for this shift:

With the progression of a series, it's common for the narrative to become more intricate, introducing new plotlines, characters, and themes.

Characters may undergo development, leading to shifts in the subjects of discussion and the focus of episodes.

As the series unfolds, different storylines begin to intertwine, resulting in a mixture of topics reflecting the merging narratives.